FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Trabajo Práctico 1 — Smalltalk

[7507/9502] Algoritmos y Programación III
Curso 2
Primer cuatrimestre de 2025

Alumno:	Mendez Archerrizo, Joaquin Ezequiel
Número de padrón:	111767
Email:	jmendeza@fi.uba.ar

Índice

1. Introducción **2**

2. Diagramas de clase **2**

3. Detalles de implementación **3**

3.1. Mostrar dos ejemplos de su código donde implementaron polimorfismo y que mecanismo utilizaron 3

4. Diagramas de secuencia **4**

1. Introducción

El presente informe reune la documentación de la solución del primer trabajo práctico de la materia Algoritmos y Programación III que consiste en desarrollar un Algoritmo que dado una criptomoneda o un token, y sus respectivos datos, calcua , dependiendo de sus datos, una prediccion sobre el valor de la criptomoneda en Pharo utilizando los conceptos del paradigma de la orientación a objetos vistos hasta ahora en el curso.

2. Diagramas de clase

Por temas de como se veia el diagrama de clases todo en un mismo diagrama, ya que se veia muy chico y era inentndible sin hacer mucho zoom. Se decidio partir el diagrama original en dos diagramas, parte 1 y parte 2. Aun asi, siento que se tiene que hacer zoom, pero mas chico que eso no creo que se pueda hacer sin perder legibilidad.

Ademas, si bien la clase Recomendador en el primer diagrama e InfoToken tambien en el segundo diagrama, era mas que nada para no repetir infromacion y en el caso del recomendador, era para mostrar la relacion con la clase AlgoPrediccionCripto.

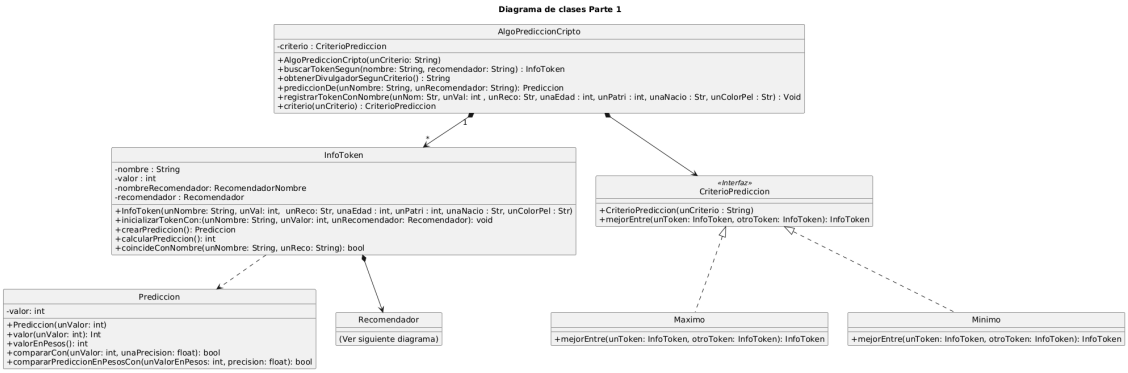

Figura 1: Diagrama numero 1 sobre las clases implementadas.

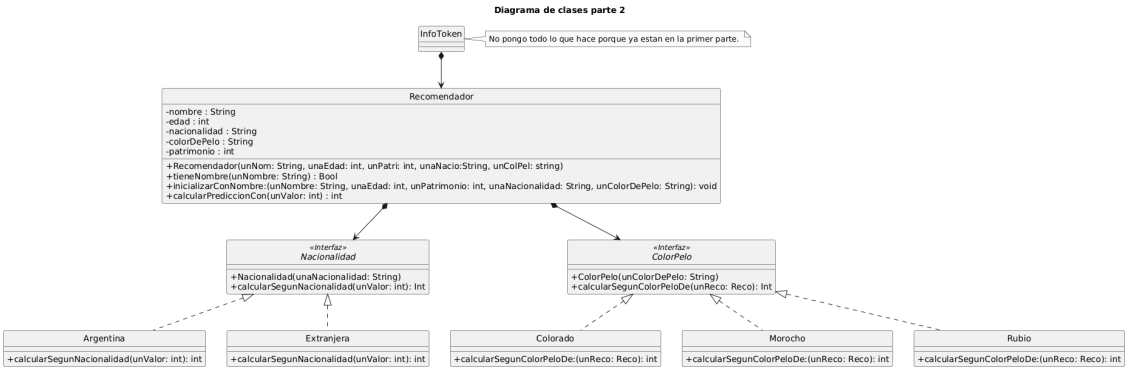

Figura 2: Diagrama numero 2 sobre las clases implementadas.

3. Detalles de implementación

3.1. Mostrar dos ejemplos de su código donde implementaron polimorfismo y que mecanismo utilizaron

En nuestra implementacion podemos decir que se aplica polimorfismo en 3 lugares concretos, concretamente con el uso de las interfaces de **Nacionalidad, ColorPelo y CriterioPrediccion**. Asi que por una cuestion de comodidad, vamos a analizar solo a **Nacionalidad y ColorPelo**, ambas interfaces de la clase Recomendador.

Entonces, en Recomendador tenemos la interfaz Nacionalidad, y dentro de esa interfaz, las subclases Argentina y Extranjera, que implementan el metodo calcularSegunNacionalidad, que se usa en recomendador (junto con ColorPelo) para calcular la prediccion de la siguiente forma:

```
calcularPrediccionCon: unValor
^ (nacionalidad calcularSegunNacionalidad: (colorDePelo calcularSegunColorPeloDe: self))
+ unValor.
```

La interfaz Nacionalidad es un ejemplo de polimorfismo porque cumple con los criterios del polimorfismo, veamos si cumple con las 3 condiciones para que sea un polimorfismo:

- La más clara es que los objetos Extranjera y Argentina responden de distinta forma al mismo mensaje, que es calcularSegunNacionalidad:. Uno lo hace de una forma y el otro de otra forma, pero ambos responden al mismo mensaje.
- Siguiendo con lo anteriormente mencionado, cada objeto maneja el comportamiento a seguir, es decir, se resuelve dentro de cada objeto (dentro de Argentina o Extranjera) cómo se calcula según la nacionalidad que tenga el recomendador.
- Por último, el objeto Recomendador, que es quien invoca esos métodos al calcular la predicción, no sabe a priori cuál clase se va a usar. Puede usarse la clase Argentina o Extranjera. Es decir, Recomendador sabe su nacionalidad, pero no sabe cómo se calcula la predicción en base a ella. Solo delega el mensaje a quien corresponda.

Luego, para el otro uso de polimorfismo, tenemos la interfaz ColorPelo, que tiene como subclases a Colorado, Rubio y Morocho. De forma análoga, podemos hacer el análisis de polimorfismo como hicimos para nacionalidad:

- Todas las subclases responden al mismo mensaje, calcularSegunColorPeloDe:, pero de forma distinta, realizando distintos cálculos para la predicción.
- Los comportamientos de cada subclase son definidos por ellas mismas. Cada una hace de forma distinta el cálculo de la predicción, según los requerimientos del problema.
- Por último, de nuevo Recomendador, que invoca a calcularSegunColorPeloDe: al hacer el cálculo de la predicción, no sabe cuál subclase se va a usar. Si bien conoce el color de pelo, no sabe cómo se calcula la predicción en base a ese color. Entonces, el recomendador no sabe nada sobre la clase que se usará; simplemente invoca el método definido en la interfaz, y esta delegará la ejecución al objeto correspondiente.

4. Diagramas de secuencia

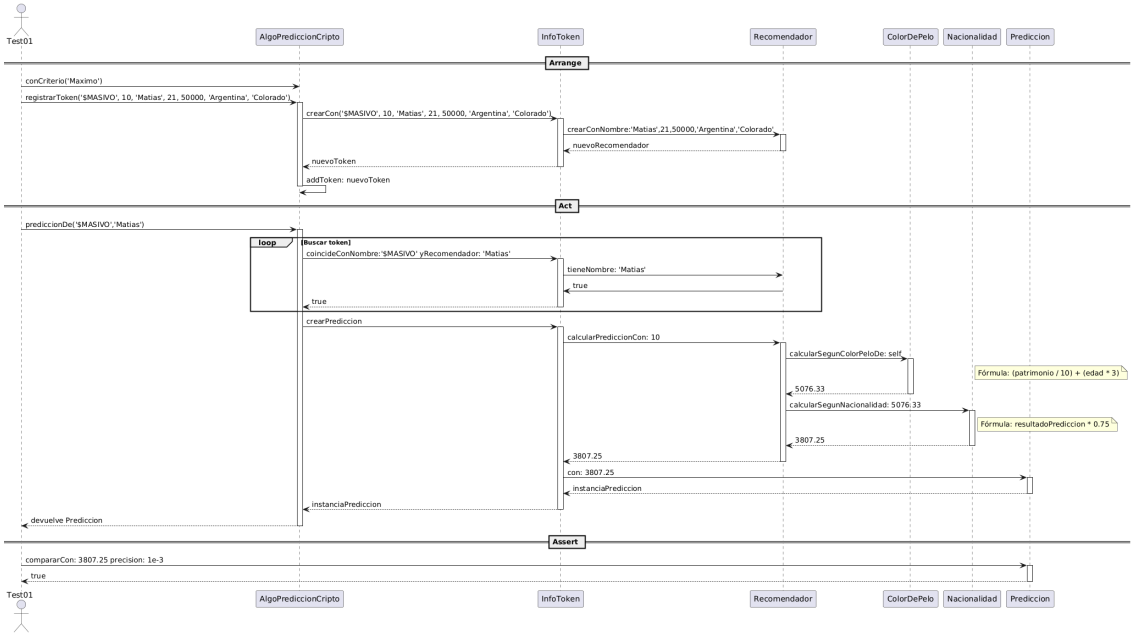

Figura 3: Diagrama de secuencia del test 01 de la clase AlgoPrediccionCripto.

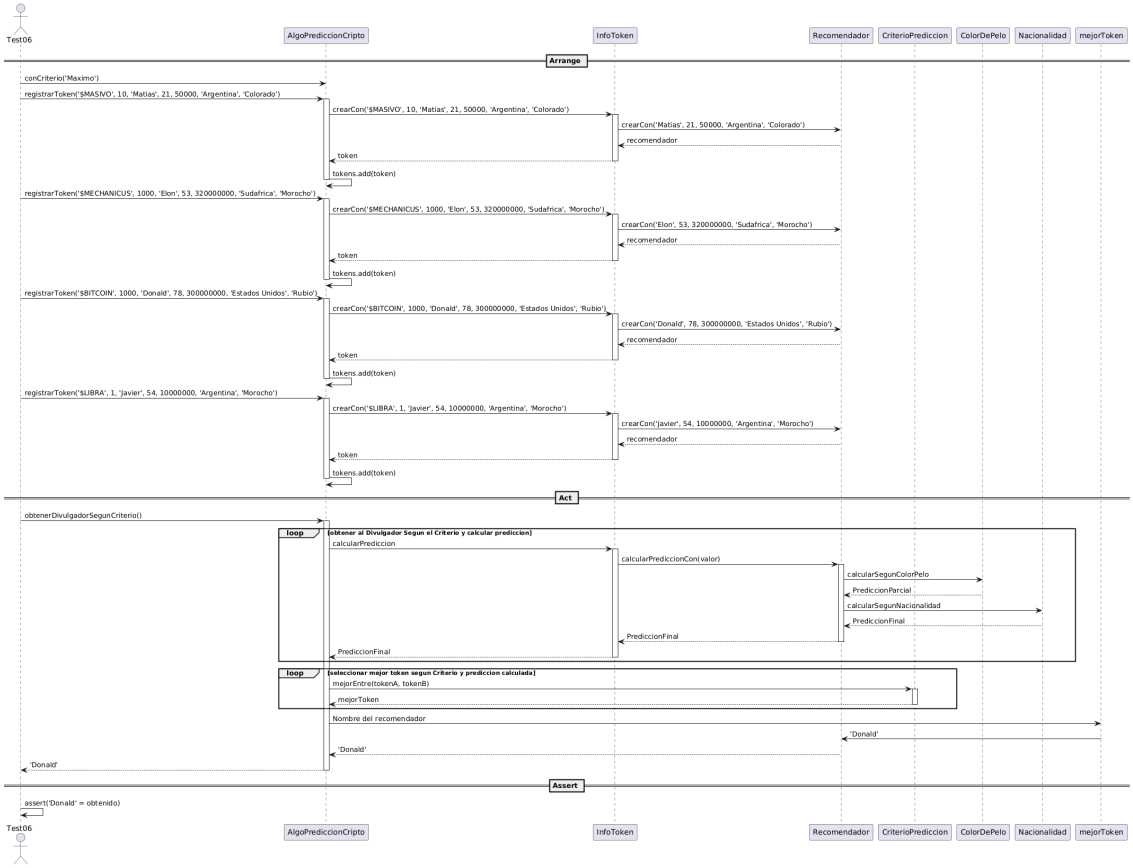

Figura 4: Diagrama de secuencia del test 06 de la clase AlgoPrediccionCripto.

Cabe restaltar que no existe una clase mejorToken propiamente, pero estaba bueno para punto de salida para cuando encontramos el mejor token que exista segun si el criterio es máximo o mínimo. Asi podemos mostrar mas graficamente que una vez que encontamos el mejor token segun el criterio que necesitamos, podemos devovler el nombre del recomendador con el mejor token posible. Y tambien que lo obtenido previo al assert es justamente el nombre del recomendador 'Donald' (que es quien se va mostrando al final del act).